



Algorithm Complexity: An Introduction



At the end of this lesson, you should be able to:

- State the concept of time complexity in algorithm implementation
- Explain the Big O notation
- Describe how Big O can be used to compare time complexity of algorithms



Recall

Algorithms

- step-by-step instructions given to computer on how to solve a given problem



- multiple possible algorithms as well as implementations
- how to compare them?
- how to evaluate which is the 'better' algorithm?



Comparison in terms of execution time



Shorter execution time $\Rightarrow$ better?

But execution time depends on many factors

- the speed of the computer
- the way the algorithm is implemented
 - pre-compute lookup table
 - loop unrolling technique
 - input data value and input data size

Instruction Steps

Count the number of instructions it takes to execute the algorithm



- independent of the computer
- more steps $\Rightarrow$ longer execution time

But the number of steps may still depend on the data involved in the computation

Example

```
def linearSearch(List, item_x):  
    for i in range(len(List)):  
        if List[i] == item_x:  
            return i  
    return -1
```

Asymptotic Behavior

More important to consider the worst case situation

- when item_x is not in the List
- number of instruction steps will be the most

As the number of entries in the List increases

- number of steps under worst case situation also increases

Note that

- 'input' data size $\equiv$ number of entries in the List

In time complexity analysis

- growth pattern of the number of steps as input data size increases indefinitely
- asymptotic behavior of running time – Big O

Example

```
def linearSearch(List, item_x):  
    for i in range(len(List)):  
        if List[i] == item_x:  
            return i  
    return -1
```

BIG O Notation

- measure and compare the time complexity of algorithms
- pattern of execution time of algorithm as input data size grows

Execution Time

- in terms of the number of instruction steps
- for worst case situation

Big O gives an upper bound on the asymptotic growth of an algorithm

Analysis of a Linear Search Algorithm

Assumption

- each line of the code statement can be executed in one step

Worst case

- item_x not in the list (of length n)
- $T(n) = 2 + 4n + 1 = 3 + 4n$

$$T(n) = 3 + 4n$$

n = 'Input' Data size $\equiv$ number of entries in the List

Example

```
def linearSearch(List, item_x):
    max_pos = len(List)
    i = 0
    while i < max_pos:
        if List[i] == item_x:
            return i
        i = i + 1
    if i == max_pos:
        return - 1
```

1
1
n times
1 x n
-
1 x n
1 x n
1

Growth Order of $T(n)$

$$T(n) = 3 + 4n$$

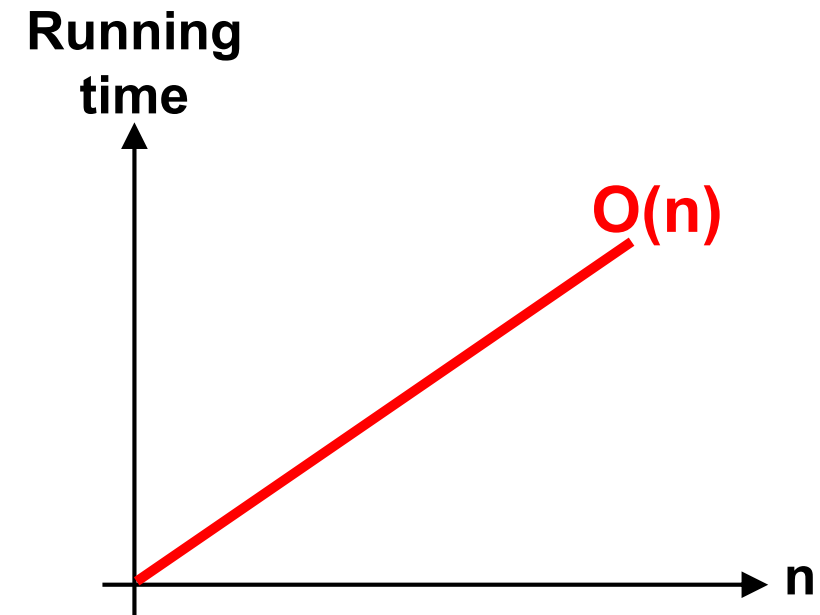
- minimum value of $T = 3$
- when $n \gg 3$, $T \rightarrow 4n$
 - e.g. for $n = 1000$
 $T = 3 + 4000 = 4003 \approx 4000$
- $T(n) \approx 4n$ for large n

Asymptotic behavior

- $T(n)$ increases proportionally with n
 - e.g. $T(n)$ doubles when n is doubled
- Growth order: $f(n) = n$

Complexity using Big O notation: $O(f(n)) = O(n)$

- Linear complexity



Program Complexity Types

Common Types of Complexity

$O(1)$

Constant complexity, where $f(n) = 1$

$O(n)$

Linear complexity, where $f(n) = n$

$O(\log n)$

Logarithmic complexity, where $f(n) = \log n$

$O(n^k)$

Polynomial complexity, where $f(n) = n^k$, with $k = \text{constant}$
e.g. $O(n^2)$ = Quadratic complexity

$O(K^n)$

Exponential complexity, where $f(n) = k^n$

Constant Complexity $O(1)$

$O(1) \equiv$ Constant Complexity

Algorithm always uses the same amount of time to execute for all inputs

Example:

*Pre-compute lookup
table-based algorithm*

Example 1

```
def lookup(position)
    return
lookupTable(position)
```

$$T(n) = 1 \equiv 1.1 \Rightarrow f(n) = 1;$$

$$O(fn) = O(1)$$

Example 2

```
def lookup(position)
    print("Entry position = " , position)
    return lookupTable(position)
```

$$T(n) = 2 \equiv 2.1 \Rightarrow f(n) = 1; O(fn) = O(1)$$

Running
time



Linear Complexity $O(n)$

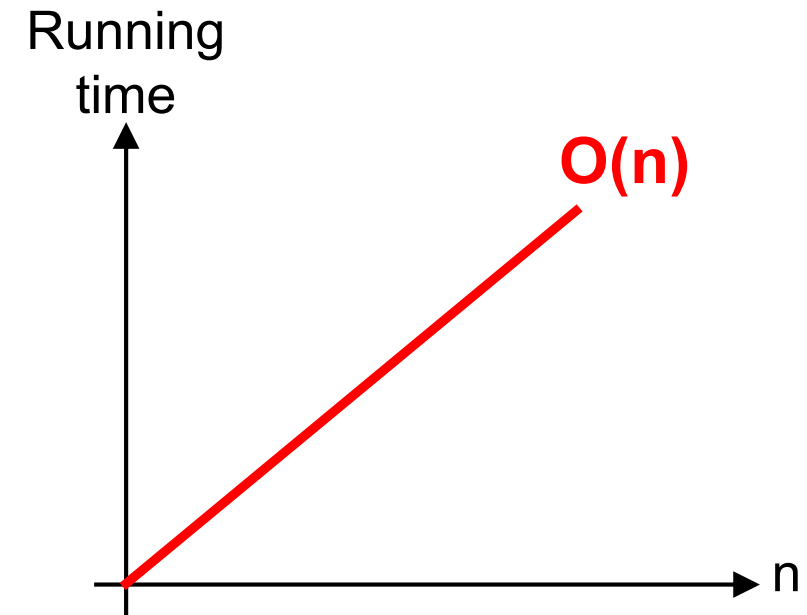
$O(n) \equiv$ Linear Complexity

Algorithm execution time increases linearly in proportion with (input) data size

Example

```
def linearSearch(List, item_x):
    for i in range(len(List)):
        if List[i] == item_x:
            return i
    return -1
```

$f(n) = \text{len(List)}$ $O(fn) = O(\text{len(List)}) = O(n)$



Polynomial Complexity $O(n^K)$

$O(n^K) \equiv$ Polynomial Complexity

Occurs for algorithm that contains nested loops

Example: Quadratic complexity $O(n^2)$

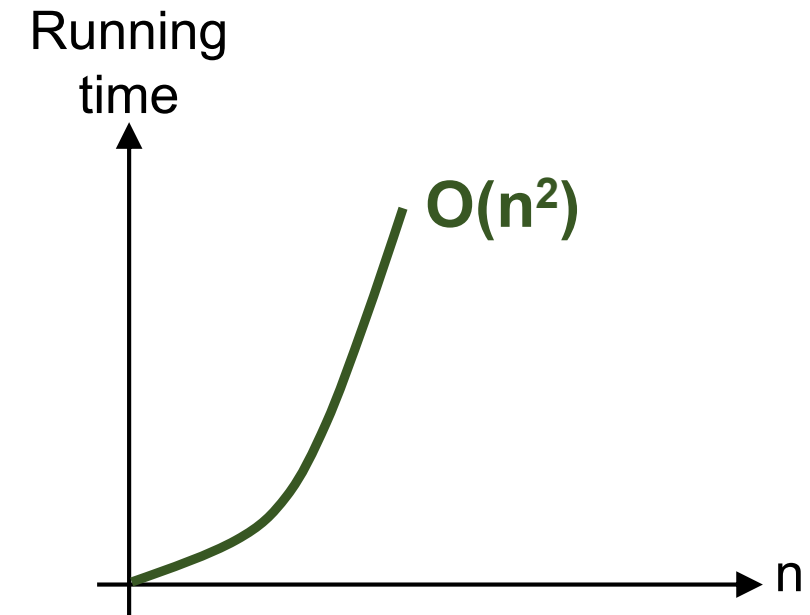
- execution time is proportional to the square of the input data size

Example: Quadratic Complexity $O(n^2)$

```
def checkDuplicate(List1, List2):
    for i in range(len(List1)):
        for j in range(len(List2)):
            if List1[i] == List2[j]:
                return List1[i]
    return -1
```

$f_{in}(n) = \text{len(List2)}$; $f_{out}(n) = \text{len(List2)}$

$O(fn) = O(f_{in}(n) \cdot f_{out}(n)) \equiv O(n \cdot n) = O(n^2)$

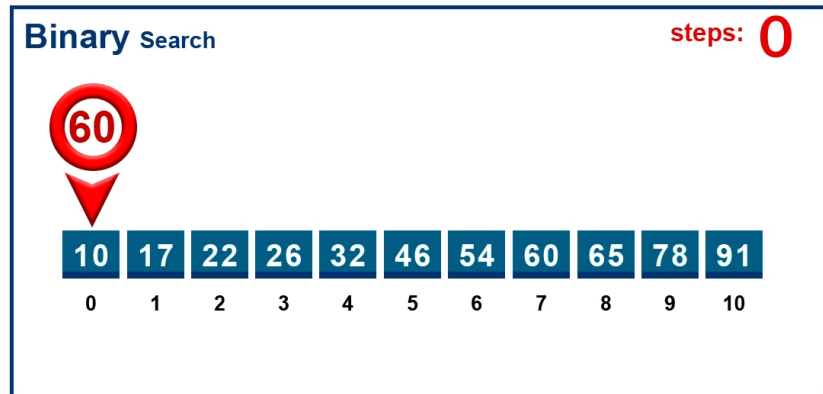


Logarithmic Complexity $O(\log n)$

$O(\log n) \equiv$ Logarithmic Complexity

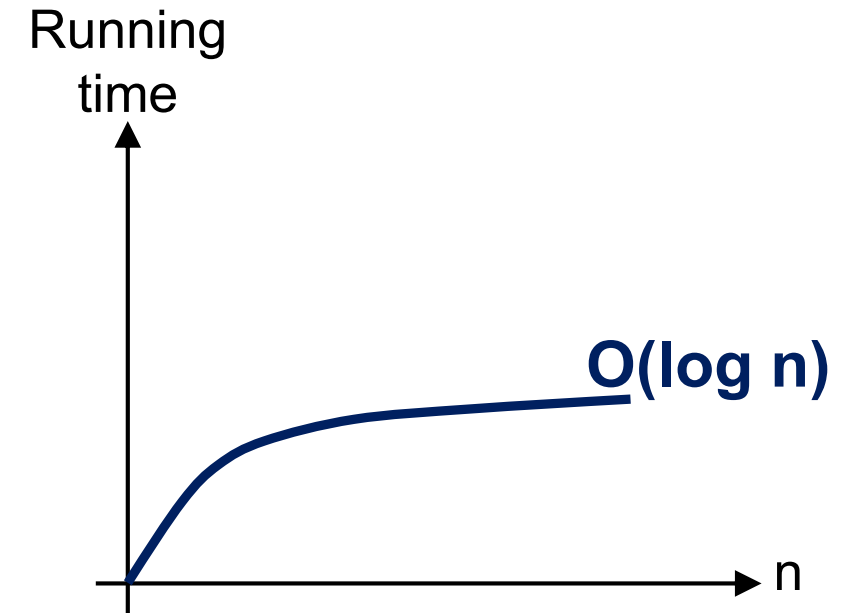
Execution time grows as the log of input

Example: Binary Search



$$(n) = \log_2(\text{len}(\text{List}))$$

$$O(\text{fn}) = O(\log_2(\text{len}(\text{List}))) \equiv O(\log n)$$



Exponential Complexity $O(k^n)$

$O(k^n) \equiv$ Exponential Complexity

Occurs for algorithm that contains recursive call

Example:

Fibonacci sequence computation

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55

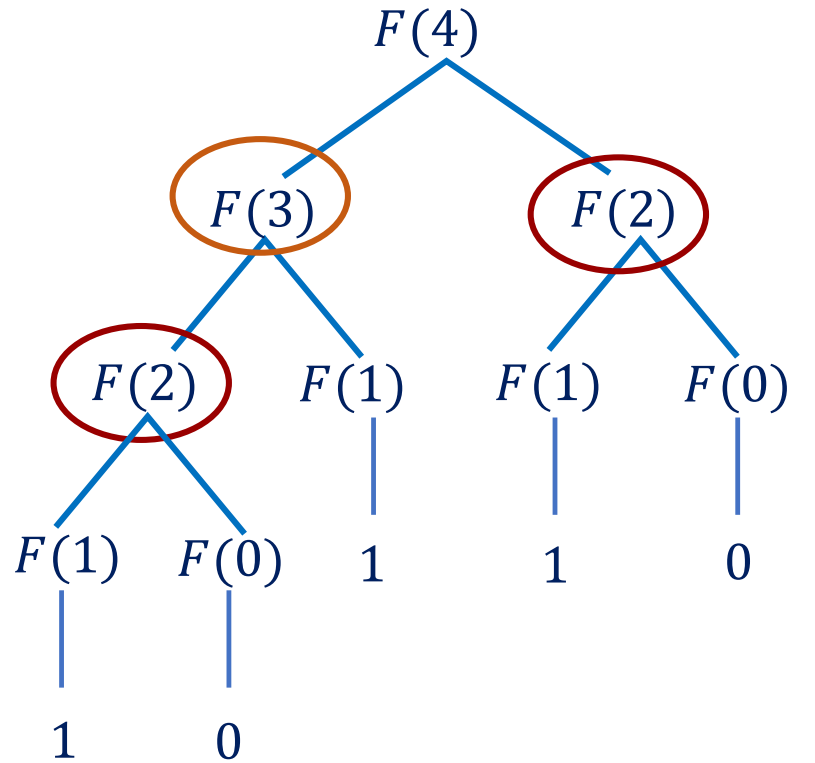
Each number can be derived based on the formula:

$$F(n) = F(n-1) + F(n-2), n \geq 2$$

Example

```
def rFib (n):  
    if (n == 0):  
        return 0  
    elif (n == 1):  
        return 1  
    elif (n > 1):  
        return (rFib(n-1) + rFib(n-2))  
    else:  
        return -1
```

Recursion Fibonacci



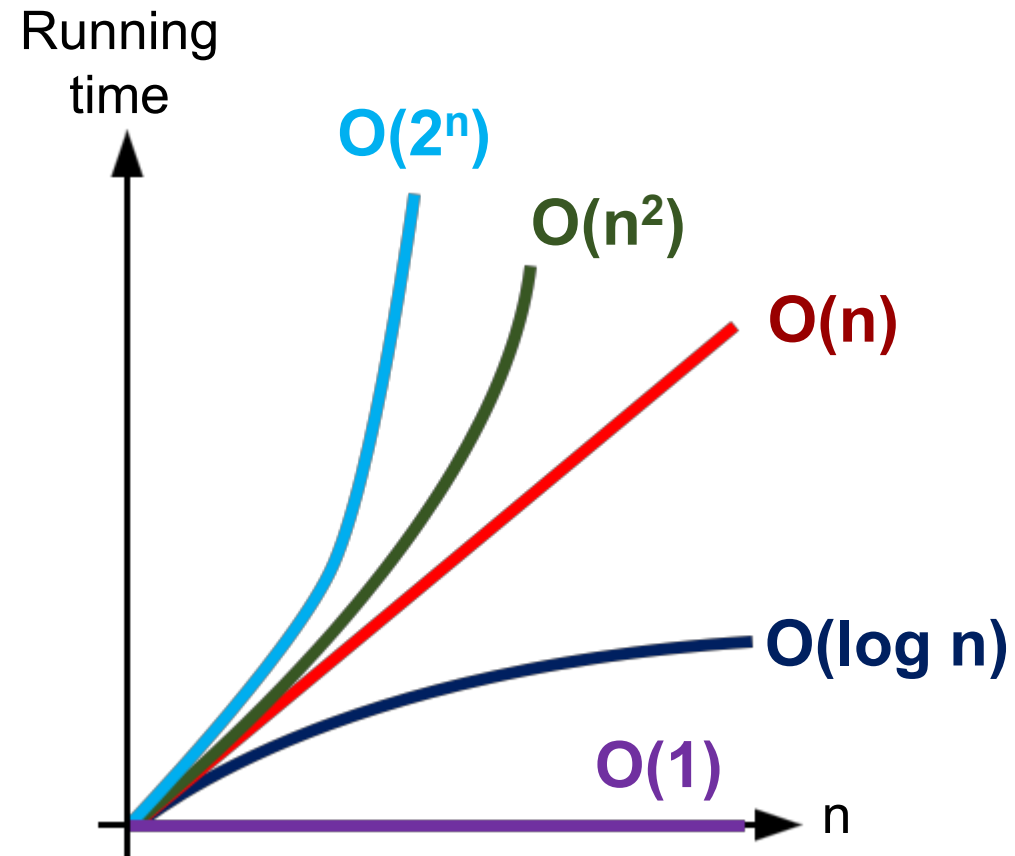
$$f(n) = 2^n$$

$$O(f(n)) = O(2^n)$$

Example

```
def rFib (n):  
    if (n == 0):  
        return 0  
    elif (n == 1):  
        return 1  
    elif (n > 1):  
        return (rFib(n-1) + rFib(n-2))  
    else:  
        return -1
```

Rate of Growth



Iteration Fibonacci

An alternative algorithm for Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55

Computation based on iteration:

$$F(n) = F(n - 1) + F(n - 2), n \geq 2$$

```
def iFib(n):  
    if (n == 0):  
        return 0  
    elif (n == 1):  
        return 1  
    elif (n == 2):  
        return 1  
    elif: # n > 2  
        #continue in next
```

```
elif :    # n > 2  
        fn0 = 0  
        fn1 = 1  
        fn2 = 1  
        for i in range(n-2):  
            fn0 = fn1  
            fn1 = fn2  
            fn2 = fn0+fn1  
        return fn2
```

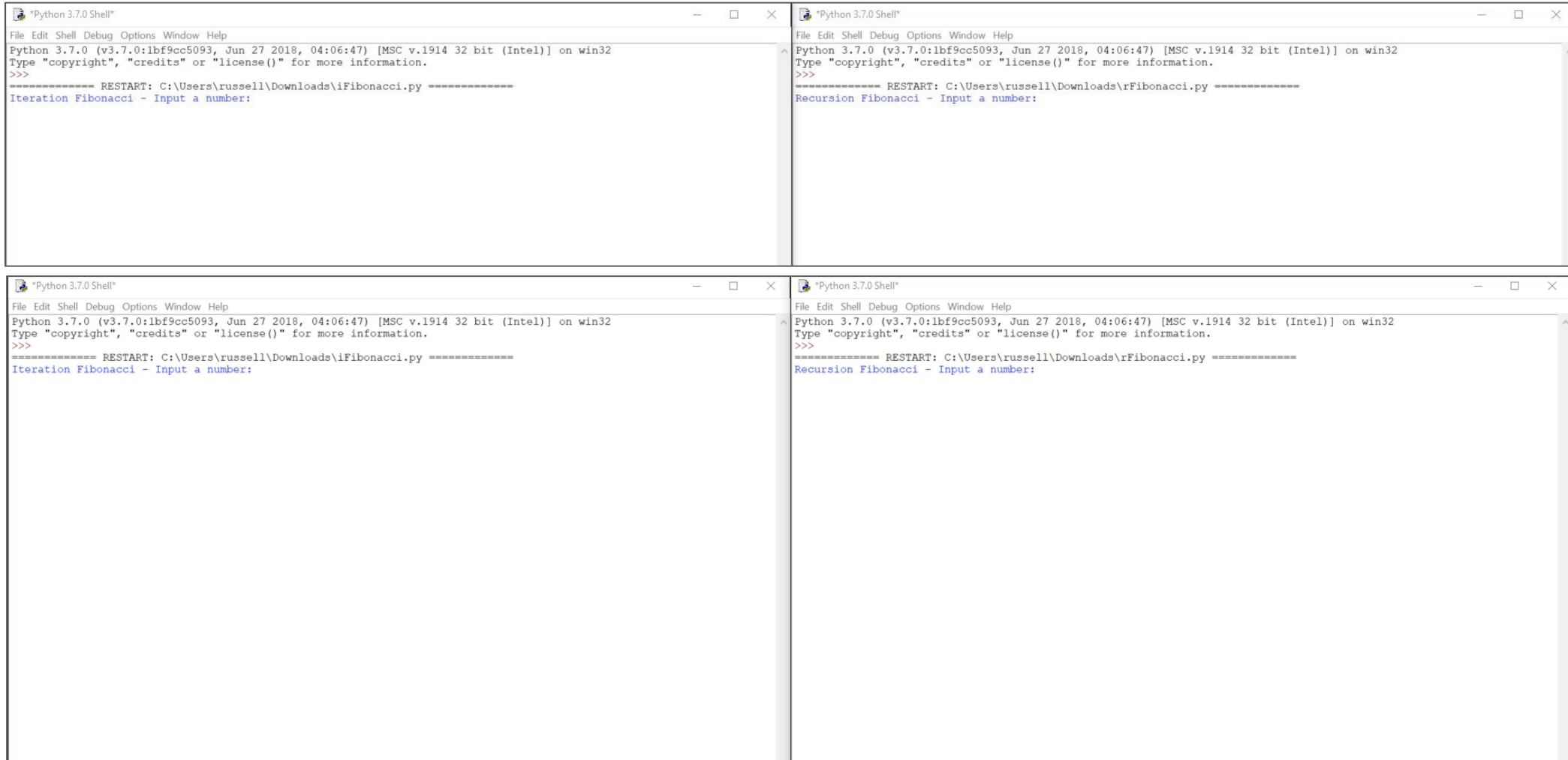
$$T(n) = 4n + 4$$

$$f(n) = n$$

$$\Rightarrow O(n)$$

i.e. Linear Complexity

Iteration vs. Recursion Fibonacci Computation



```
*Python 3.7.0 Shell*
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\russell\Downloads\iFibonacci.py =====
Iteration Fibonacci - Input a number:

*Python 3.7.0 Shell*
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\russell\Downloads\rFibonacci.py =====
Recursion Fibonacci - Input a number:
```

Iteration vs. Recursion Fibonacci Computation (Cont'd)

```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\russell\Downloads\IFibonacci.py =====
Iteration Fibonacci - Input a number:
```

```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\russell\Downloads\rfibonacci.py =====
Recursion Fibonacci - Input a number:
```

```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\russell\Downloads\IFibonacci.py =====
Iteration Fibonacci - Input a number:
```

```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\russell\Downloads\rfibonacci.py =====
Recursion Fibonacci - Input a number:
```

Algorithm complexity can be analyzed

- time complexity

Big O

- worst case analysis
- order of growth pattern as input size grows

Asymptotic behavior of algorithms

- useful for comparing and classifying algorithms

Different algorithms for same problem

- compared based on Big O

References for Images

No.	Slide No.	Image	Reference
1	4		Question problem [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/question-problem-think-thinking-622164/ .
2	4		Search [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/database-search-database-search-icon-2797375/ .
3	5		Search [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/stopwatch-timer-watch-seconds-34107/ .



Pattern Recognition in Python

Lesson Objectives



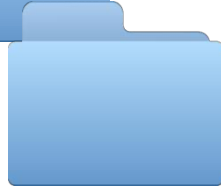
At the end of this lesson, you should be able to:

- Recognize similar patterns in problems
- Apply pattern recognition in Python programming language

Topic Outline



Iterative Accumulation



Practices on Pattern Recognition

Iterative Accumulation is a very common operation.

It accumulates **target values** iteratively.

Iterative Accumulation: Example



How to calculate the result of
 $1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10$?

Target values in each iteration:

- Iteration 1: 1
- Iteration 2: 2
- Iteration 3: 3
- $\vdots$
- Iteration 10: 10

accumulate

55

Iterative Accumulation: Python Code

Three Important Elements

result variable →
(to store the
accumulation result)

```
n = 10
result = 0

for i in range(1, n+1):
    result = result + i

print(result)
```

for loop

target value
(in each iteration)

Problem 1



How to calculate the result of

$$1/1 + 1/2 + 1/3 + 1/4 + 1/5 + 1/6 + 1/7 + 1/8 + 1/9 + 1/10?$$

Target values in each iteration:

- Iteration 1: $1/1$
- Iteration 2: $1/2$
- Iteration 3: $1/3$
- $\vdots$
- Iteration 10: $1/10$

generalize

Iteration i : $1/i$

Problem 1: Python Code



How to calculate the result of

$$1/1 + 1/2 + 1/3 + 1/4 + 1/5 + 1/6 + 1/7 + 1/8 + 1/9 + 1/10?$$

Three Important Elements

- **result variable**
- **for loop**
- **target value**

Iteration i: **1/i**

```
n = 10
result = 0

for i in range(1,n+1):
    result = result + 1/i

print(result)
```

Problem 2



How to calculate the result of

$$1/(1*2) + 1/(2*3) + 1/(3*4) + 1/(4*5) + 1/(5*6) + 1/(6*7) + 1/(7*8) + 1/(8*9) + 1/(9*10) + 1/(10*11)?$$

Target values in each iteration:

- Iteration 1: $1/(1*2)$
- Iteration 2: $1/(2*3)$
- Iteration 3: $1/(3*4)$
- $\vdots$
- Iteration 10: $1/(10*11)$

generalize

Iteration i: $1/(i*(i+1))$

Problem 2: Python Code



How to calculate the result of

$1/(1*2) + 1/(2*3) + 1/(3*4) + 1/(4*5) + 1/(5*6) + 1/(6*7) + 1/(7*8) + 1/(8*9) + 1/(9*10) + 1/(10*11)?$

Three Important Elements

- **result variable**
- **for loop**
- **target value**

Iteration i: **$1/(i*(i+1))$**

```
n = 10
result = 0

for i in range(1,n+1):
    result = result + 1/(i*(i+1))

print(result)
```

Problem 3



How to calculate the result of

$$0/1 + 1/2 + 2/3 + 3/4 + 4/5 + 5/6 + 6/7 + 7/8 + 8/9 + 9/10?$$

Target values in each iteration:

- Iteration 1: 0/1
- Iteration 2: 1/2
- Iteration 3: 2/3
- $\vdots$
- Iteration 10: 9/10

generalize

Iteration i: (i-1)/i

Problem 3: Python Code



How to calculate the result of

$$0/1 + 1/2 + 2/3 + 3/4 + 4/5 + 5/6 + 6/7 + 7/8 + 8/9 + 9/10?$$

Three Important Elements

- **result variable**
- **for loop**
- **target value**

Iteration i: **$(i-1)/i$**

```
n = 10
result = 0

for i in range(1,n+1):
    result = result + (i-1)/i

print(result)
```

Problem 4



How to calculate the result of

$$1/9 + 2/8 + 3/7 + 4/6 + 5/5 + 6/4 + 7/3 + 8/2 + 9/1?$$

Target values in each iteration:

- Iteration 1: $1/9$
- Iteration 2: $2/8$
- Iteration 3: $3/7$
- $\vdots$
- Iteration 9: $9/1$

generalize

Iteration i : $i/(10-i)$

Problem 4: Python Code



How to calculate the result of

$$1/9 + 2/8 + 3/7 + 4/6 + 5/5 + 6/4 + 7/3 + 8/2 + 9/1?$$

Three Important Elements

- **result variable**
- **for loop**
- **target value**

Iteration i: **$i/(10-i)$**

```
n = 9
result = 0

for i in range(1,n+1):
    result = result + i/(10-i)

print(result)
```

Problem 5



How to calculate the result of

$$\frac{3}{5} + \frac{4}{6} + \frac{5}{7} + \frac{6}{8} + \frac{7}{9} + \frac{8}{10} + \frac{9}{11} + \frac{10}{12} + \frac{11}{13} + \frac{12}{14}?$$

Target values in each iteration:

- Iteration 1: $\frac{3}{5}$
- Iteration 2: $\frac{4}{6}$
- Iteration 3: $\frac{5}{7}$
- $\vdots$
- Iteration 10: $\frac{12}{14}$

generalize

Iteration i: $\frac{(i+2)}{(i+4)}$

Problem 5: Python Code



How to calculate the result of

$$3/5 + 4/6 + 5/7 + 6/8 + 7/9 + 8/10 + 9/11 + 10/12 + 11/13 + 12/14?$$

Three Important Elements

- **result variable**
- **for loop**
- **target value**

Iteration i: **$(i+2)/(i+4)$**

```
n = 10
result = 0

for i in range(1,n+1):
    result = result + (i+2)/(i+4)

print(result)
```

Problem 6



How to calculate the result of

$$1 - 2 + 3 - 4 + 5 - 6 + 7 - 8 + 9 - 10?$$

Target values in each iteration:

- Iteration 1: +1
- Iteration 2: -2
- Iteration 3: +3
- Iteration 4: -4
- ⋮
- Iteration 9: +9
- Iteration 10: -10

generalize

Iteration i

- i is odd: $+i$
- i is even: $-i$

Problem 6: Python Code



How to calculate the result of

$1 - 2 + 3 - 4 + 5 - 6 + 7 - 8 + 9 - 10?$

Three Important Elements

- **result variable**
- **for loop**
- **target value**

Iteration i

- **i is odd: +i**
- **i is even: -i**

```
n = 10
result = 0

for i in range(1, n+1):
    if i % 2 == 1:
        result = result + i
    else:
        result = result - i

print(result)
```

Problem 7



How to calculate the result of

$$0/1 - 1/2 + 2/3 - 3/4 + 4/5 - 5/6 + 6/7 - 7/8 + 8/9 - 9/10?$$

Target values in each iteration:

- Iteration 1: $+0/1$
- Iteration 2: $-1/2$
- Iteration 3: $+2/3$
- Iteration 4: $-3/4$
- $\vdots$
- Iteration 9: $+8/9$
- Iteration 10: $-9/10$

generalize

Iteration i

- i is odd: $+(i-1)/i$
- i is even: $-(i-1)/i$

Problem 7: Python Code



How to calculate the result of

$$0/1 - 1/2 + 2/3 - 3/4 + 4/5 - 5/6 + 6/7 - 7/8 + 8/9 - 9/10?$$

Three Important Elements

- **result variable**
- **for loop**
- **target value**

Iteration i

- **i is odd: $+(i-1)/i$**
- **i is even: $-(i-1)/i$**

```
n = 10
result = 0

for i in range(1,n+1):
    if i%2 == 1:
        result = result + (i-1)/i
    else:
        result = result - (i-1)/i

print(result)
```

In this lesson, we have learned:

- Iterative Accumulation
- Application of Pattern Recognition in Python

References for Images

No.	Slide No.	Image	Reference
1	6		Question problem [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/question-problem-think-thinking-622164/ .



Pattern Recognition



At the end of this lesson, you should be able to:

- Describe the concept of pattern recognition
- Apply pattern recognition to solve complex problems



What is a Pattern?



What is a Pattern Recognition?



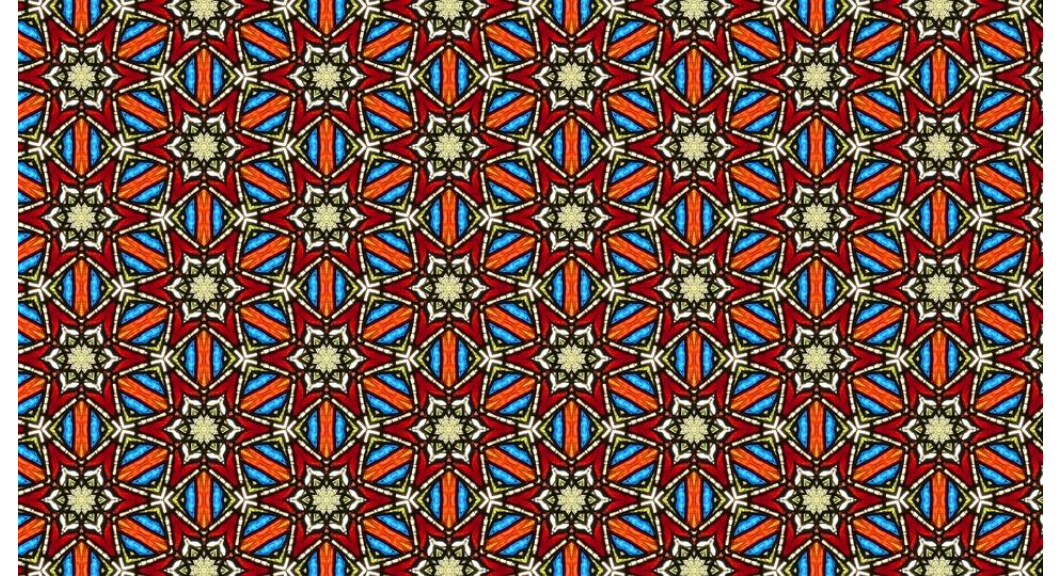
Why is Pattern Recognition Important?



How to recognize Patterns?

What is a Pattern?

- A **pattern** is a discernible regularity.
 - The elements of a pattern repeat in a **predictable manner**.
- In computational thinking, a **pattern** is the spotted **similarities** and **common differences** between problems.



Example: 1, 4, 7, 10, 13, 16, 19, 22, 25, ...

$$\downarrow$$

$$A_n = A_{n-1} + 3$$

What is Pattern Recognition

Pattern Recognition is one of the four cornerstones of Computational Thinking.

It involves finding the similarities or patterns among small, decomposed problems, which can help us solve complex problems more efficiently.



Why is Pattern Recognition Important?

- Patterns make problems simpler and easy to solve.
- Problems are easier to solve when they share patterns, we can use the same problem-solving solution wherever the pattern exists.
- The more patterns we can find, the easier and quicker our problem solving will be.

How to Recognize Patterns?

Pattern recognition is a process involving the following steps:

- 1 Identifying common elements or features in problems.
- 2 Identifying and interpreting common differences between problems.
- 3 Identifying Individual elements within problems.
- 4 Describing patterns that have been identified.
- 5 Making predictions based on identified patterns.

Example 1



Suppose we already know that: $1 + 2 + 3 + 4 + \dots + 100 = 5050$

How do we quickly calculate the result of $2 + 4 + 6 + 8 + \dots + 200$?

Answer

$$2 + 4 + 6 + 8 + \dots + 200 = 2 * (1 + 2 + 3 + 4 + \dots + 100)$$



$$= 2 * 5050$$

Example 2



Suppose we already know that: $1 + 2 + 3 + 4 + \dots + 100 = 5050$

How do we quickly calculate the result of

$101 + 102 + 103 + 104 + \dots + 200$?

Answer

$$\begin{aligned} & 101 + 102 + 103 + 104 + \dots + 200 \\ &= \underbrace{100 + 1}_{100 + 1} + \underbrace{100 + 2}_{100 + 2} + \underbrace{100 + 3}_{100 + 3} + \underbrace{100 + 4}_{100 + 4} + \dots + \underbrace{100 + 100}_{100 + 100} \\ &= \underbrace{(100 + 100 + 100 + 100 + \dots + 100)}_{100 \text{ times}} + (1 + 2 + 3 + 4 + \dots + 100) \\ &= \boxed{100 * 100} + \boxed{5050} \end{aligned}$$

In this lesson, we have learned:

- What Pattern Recognition is
- Importance of Pattern Recognition
- Steps in Recognizing Patterns

References for Images

No.	Slide No.	Image	Reference
1	5		Survey icon [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/survey-icon-survey-icon-2316468/ .
2	5		Pattern [Online Image]. Retrieved July 2, 2018 from https://pixabay.com/en/pattern-stained-glass-church-2661920/ .
3	6		Twinkle [Online Image]. Retrieved July 2, 2018 from https://www.deviantart.com/fleur555/art/Twinkle-71905382 .
4	9, 10		Question problem [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/question-problem-think-thinking-622164/ .